



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Software Platforms Course

Distributed Document Search and Analysis Platform

Project Documentation

Authors:

Fabio Baldan, Giancarlo Padoan, Leonardo Ongaro

January 2026

Contents

1	Introduction	1
2	Project Objectives	1
3	Project Requirements	1
4	Evaluation Criteria	1
5	Project Structure	2
6	4+1 view model	2
6.1	Logical View	2
6.1.1	Class diagrams	2
6.2	Process View	4
6.2.1	Activity diagrams	4
6.2.2	Sequence diagrams	6
6.3	Implementation View	7
6.3.1	Component diagram	7
6.3.2	Package diagram	8
6.4	Deployment View	9
6.4.1	Use Case View	9
6.4.2	Use case diagram	9
6.4.3	User stories	10
7	Architecture Justification	11
7.1	Purpose and Context	11
7.2	Architectural Overview	11
7.3	Roles and Justifications of Core Design Choices	11
7.3.1	Elasticsearch as Search Backend	11
7.3.2	MongoDB for Query Caching and Analytical Results	11
7.3.3	MALLET for Topic Modeling	12
7.3.4	Custom Importer	12
7.3.5	Reverse Proxy and Controlled Exposure	12
7.3.6	Swagger API generated docs	12
7.3.7	Applying the Inference Service After the Search Phase	12
7.3.8	Custom trainer	13
7.3.9	Separated Owilix Dockerfile	13
7.3.10	Data duplication	13
7.4	Architectural Benefits	14
7.5	Deployment Considerations	14
7.6	Conclusion	14
8	Our Project vs. the Competition	14

1 Introduction

This project was developed as part of the UniPD's **Software Platforms** course.

The objective is to design and implement a distributed platform that collects documents from one or more online sources, identifies documents related to a specific topic (e.g. **science and technology**), stores them, makes them searchable, and extracts useful representations for expert users.

The system follows a **service-oriented architecture**, emphasizing modularity, reproducibility, and documented design patterns.

2 Project Objectives

- Collect documents from external online sources;
- Identify and filter documents related to science and technology;
- Store documents in a persistent database;
- Provide search and indexing capabilities;
- Perform topic modeling to extract meaningful representations;
- Support expert users through structured data access and analysis.

3 Project Requirements

- Use **multiple services** (microservice-oriented approach);
- Use **Docker** for containerized deployment (recommended);
- Services are implemented primarily in **Java**;
- Apply and document **design patterns and architectural choices**;
- Architecture documentation is a **core evaluation component**.

4 Evaluation Criteria

The project is evaluated based on:

- **Documentation**;
- **Code quality**;
- **Reproducibility**;
- **Final presentation**;
- Every group member must actively participate;
- A slide-based presentation (e.g., PowerPoint) is recommended.

5 Project Structure

The following diagram illustrates the system pipeline:

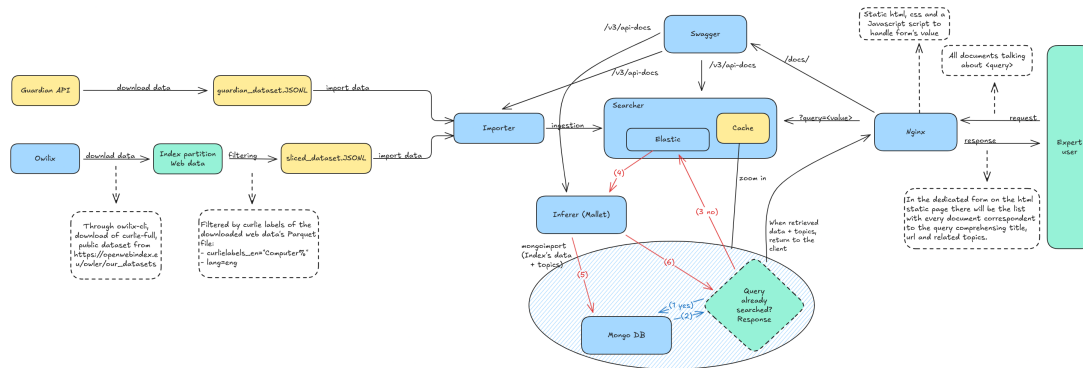


Figure 1: System pipeline overview. Blue elements represent services instantiated in separate Docker containers; yellow elements represent auxiliary components such as scripts or files; green blocks represent logical groupings introduced for clarity.

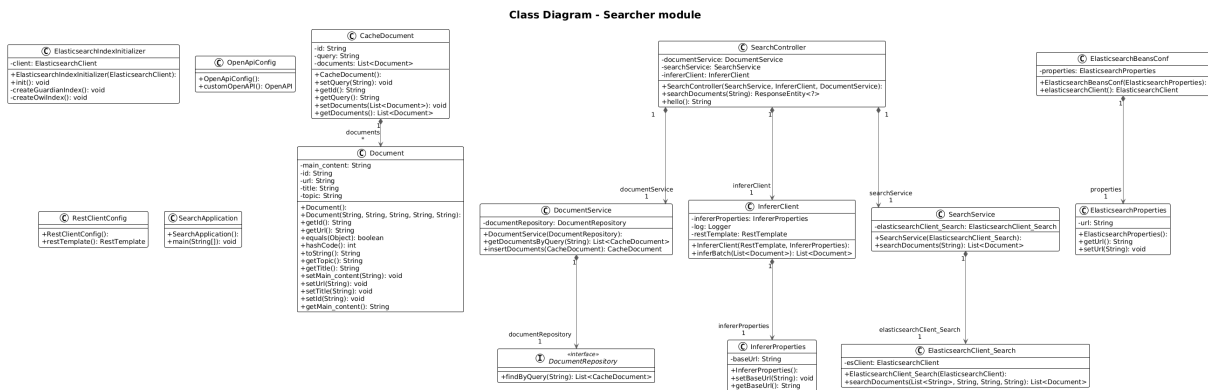
6 4+1 view model

Here is a brief recap of the 4+1 architectural view model and its goals. The model provides a comprehensive framework for documenting software architecture through multiple perspectives, each addressing different stakeholder concerns.

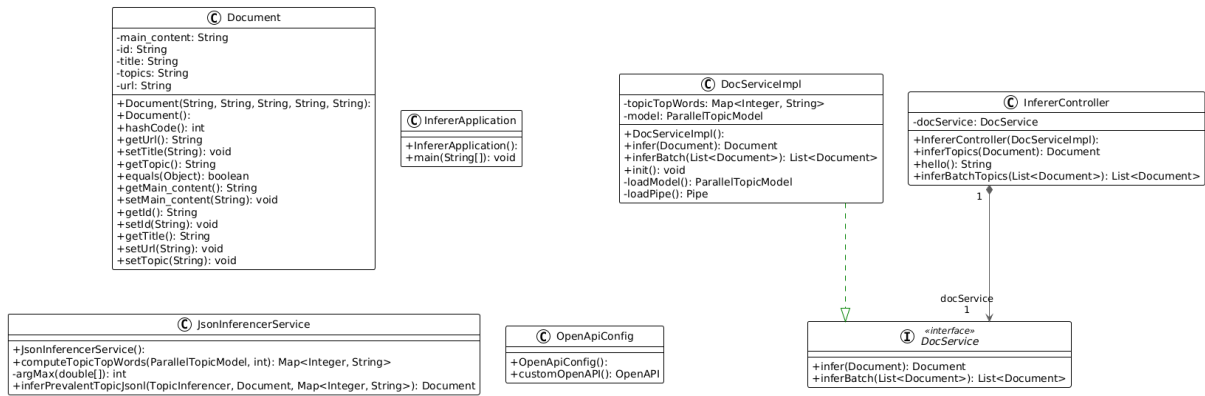
6.1 Logical View

It primarily serves **analysts** and **designers** who need to understand the system's functionality.

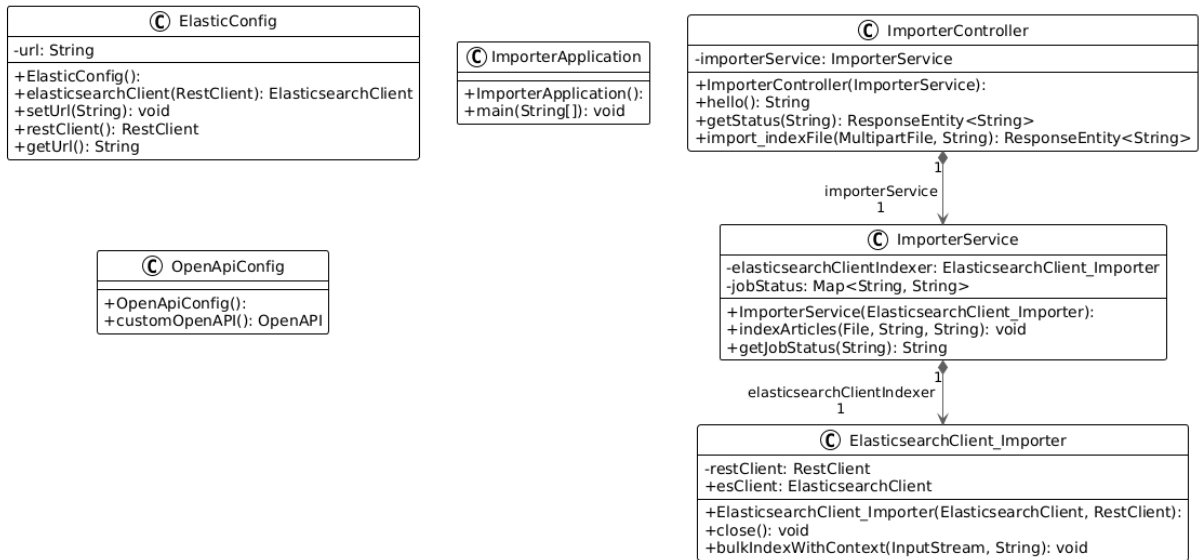
6.1.1 Class diagrams



Class Diagram - Inferer module



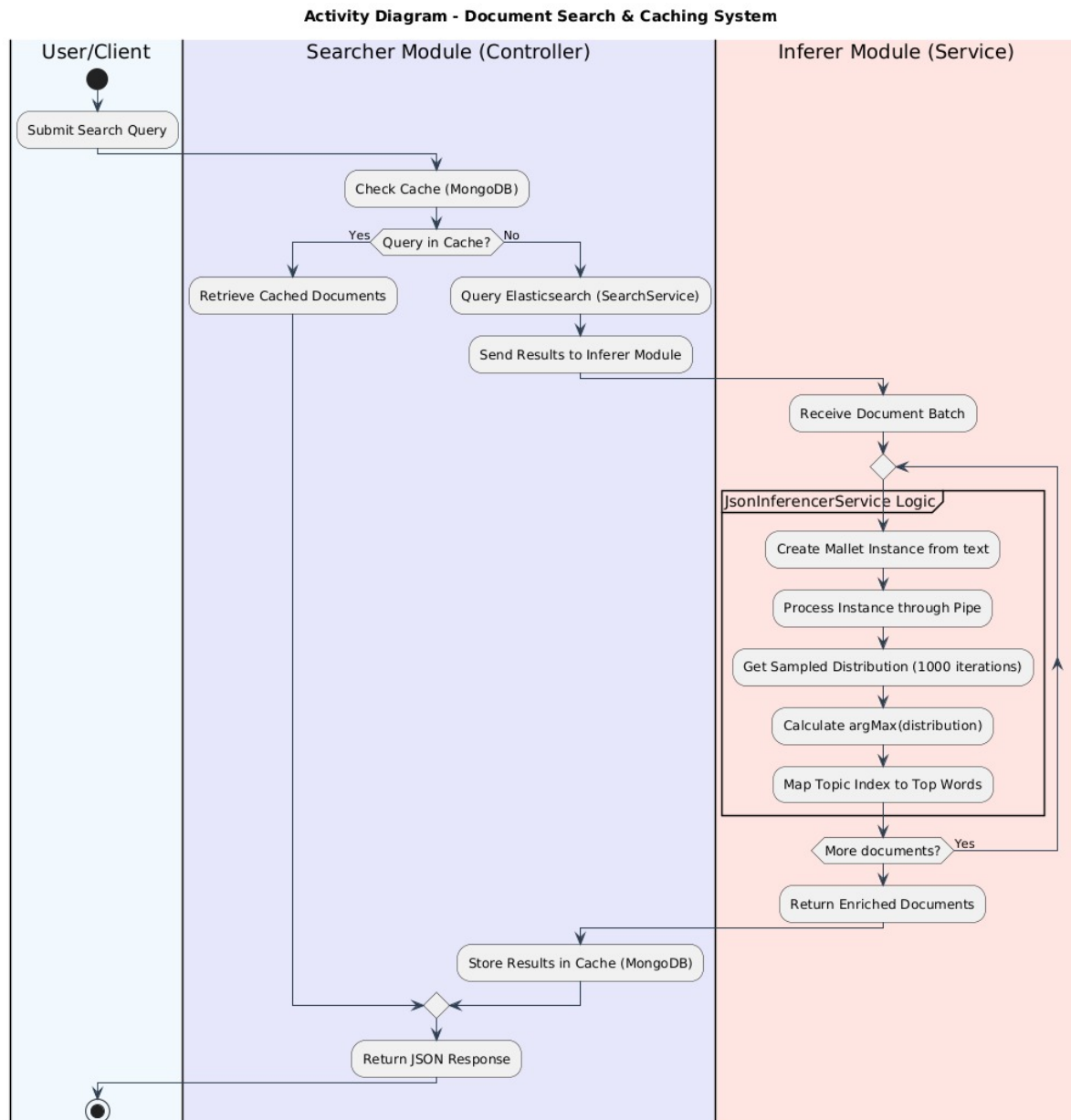
Class Diagram - Importer module



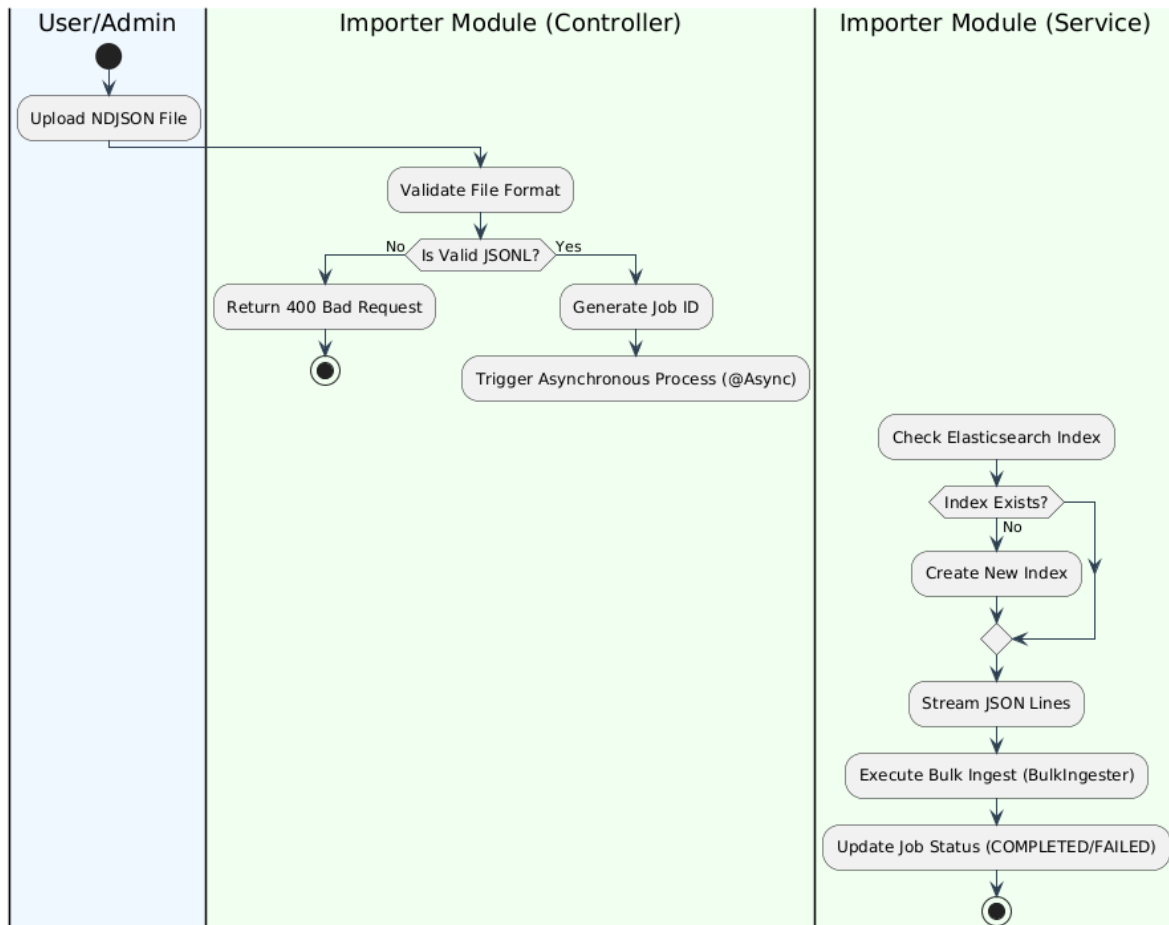
6.2 Process View

This is particularly valuable for **integrators** and **engineers** concerned with system performance, scalability, and throughput.

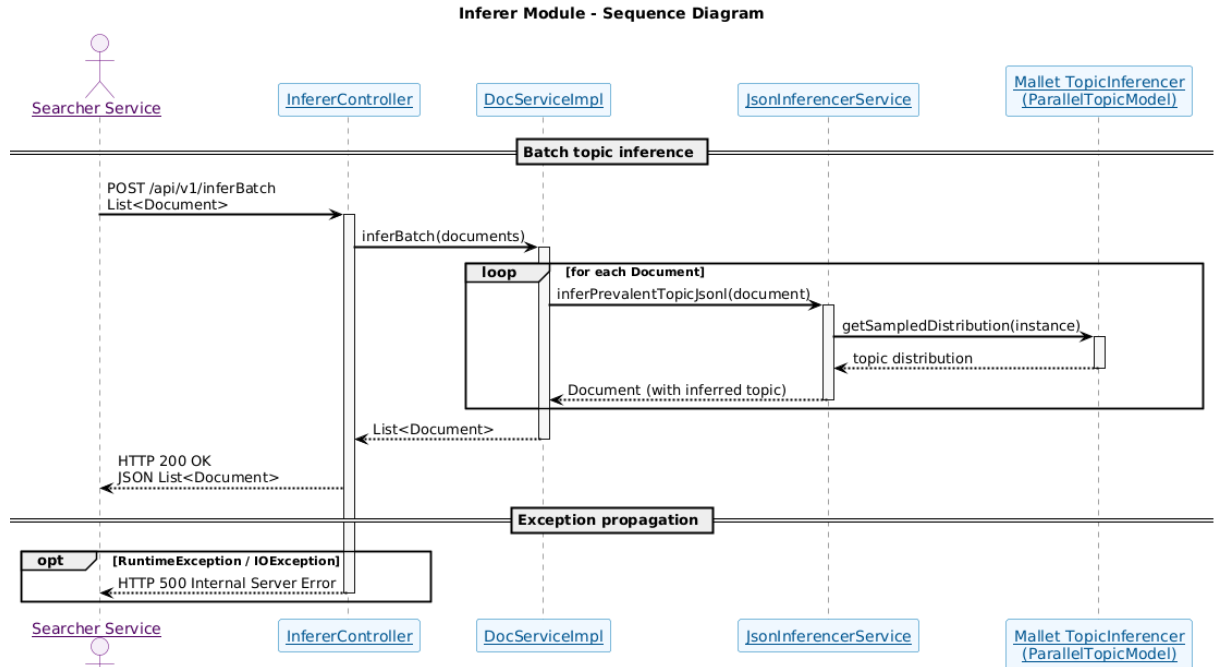
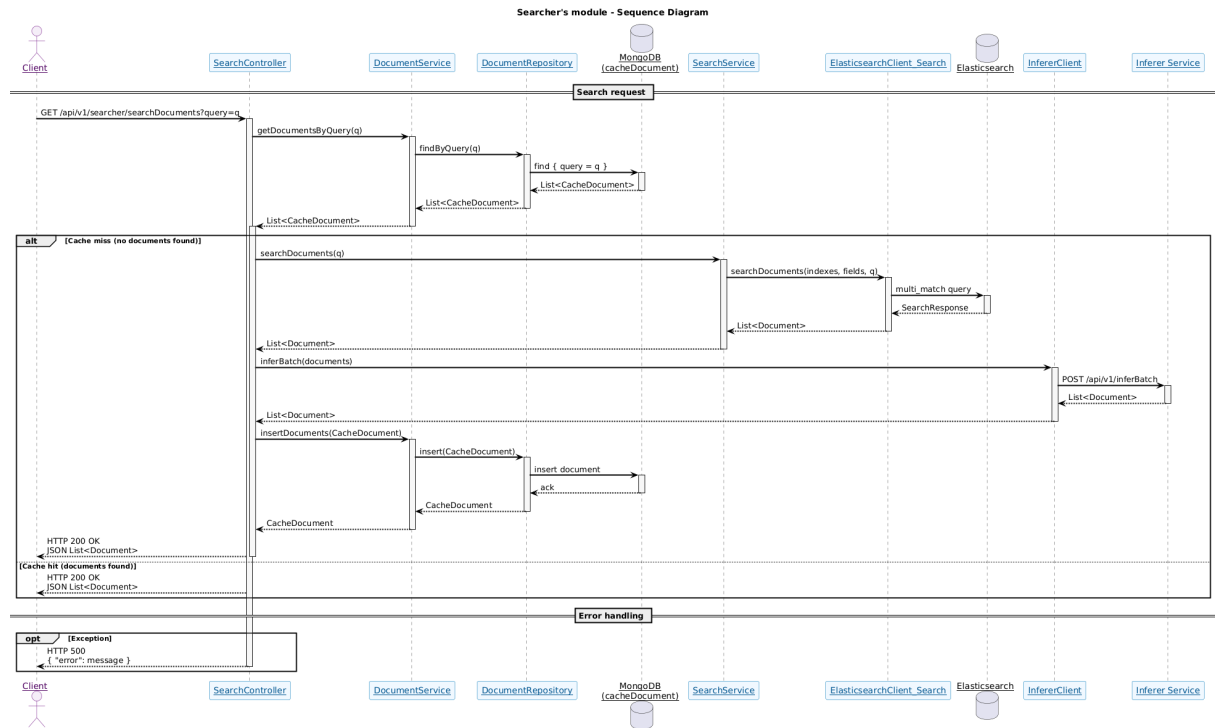
6.2.1 Activity diagrams

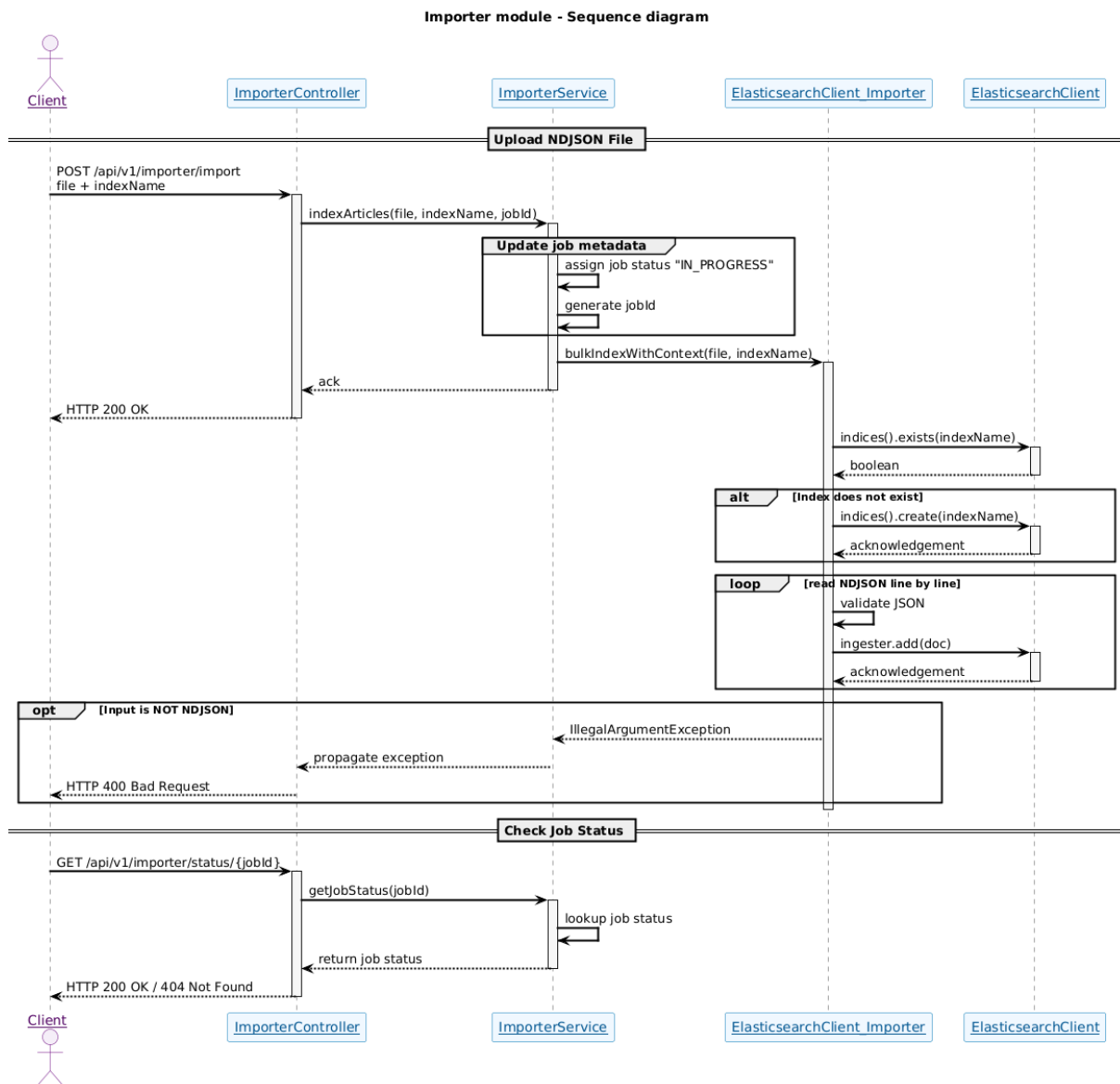


Activity Diagram - Importing System



6.2.2 Sequence diagrams



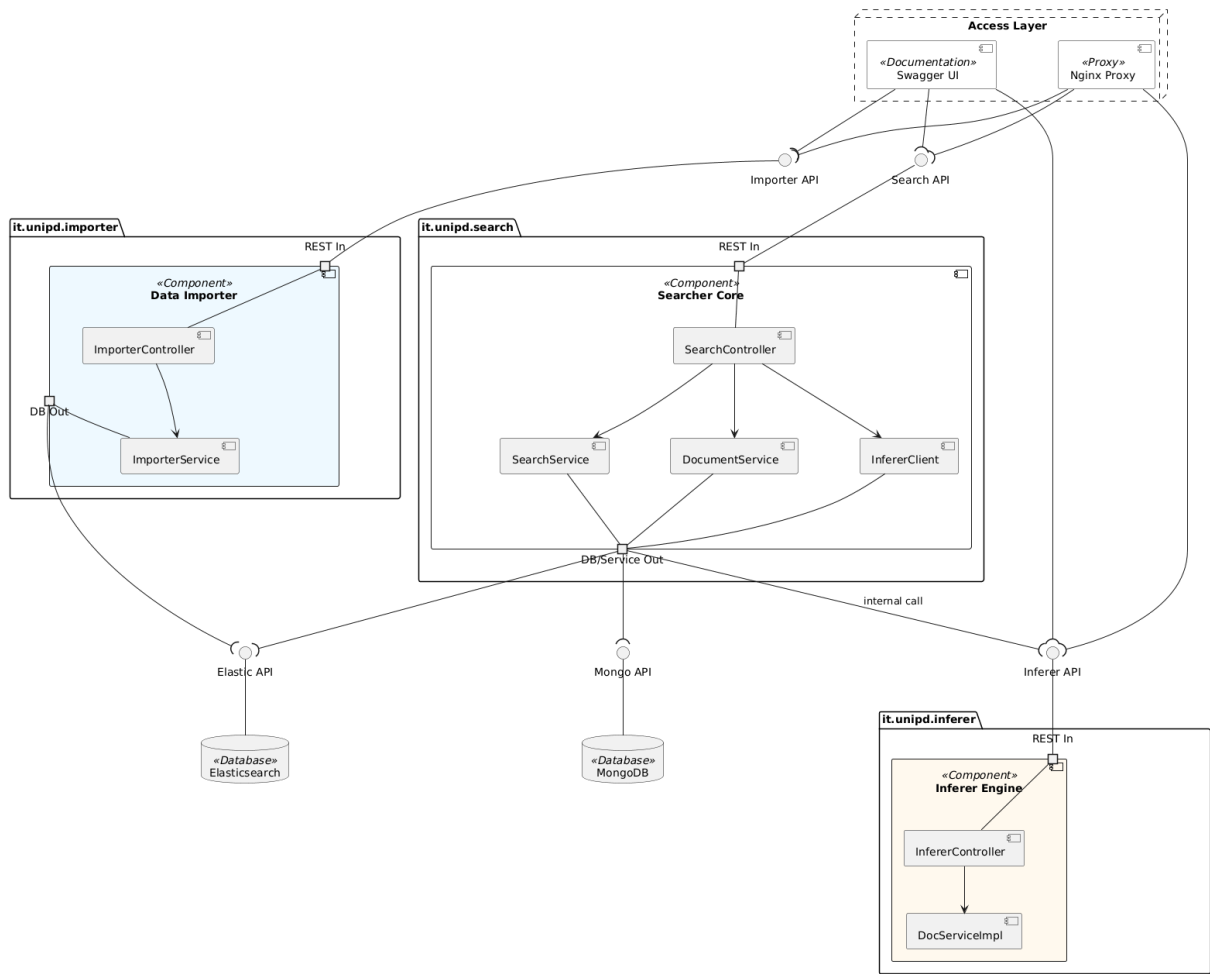


6.3 Implementation View

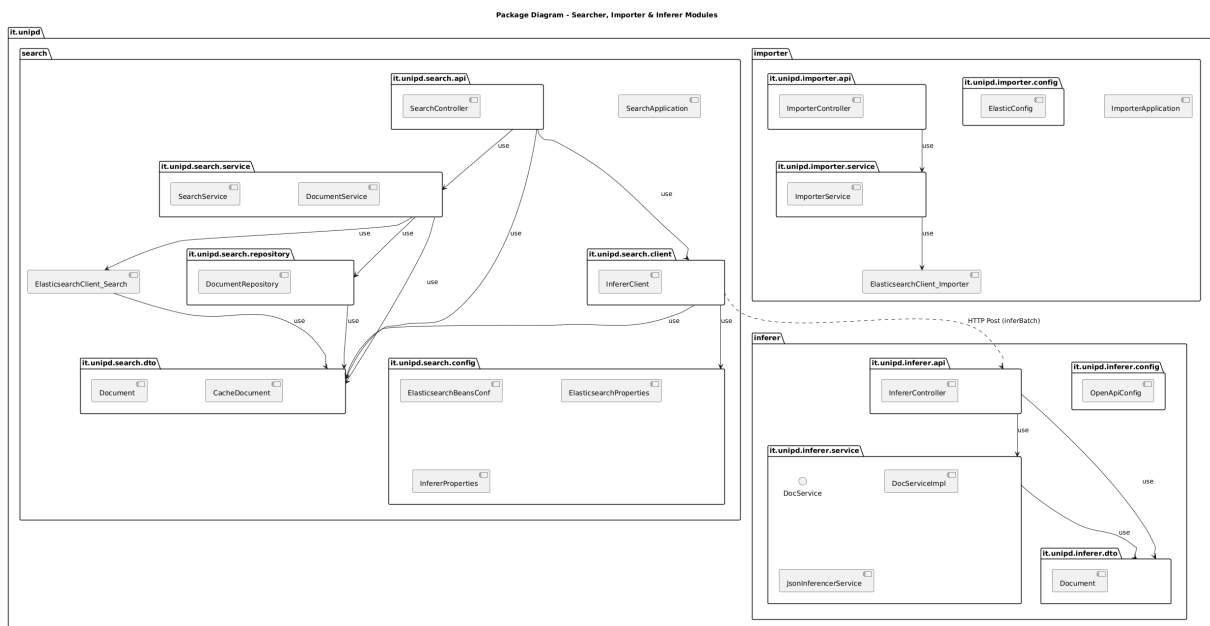
It guides **developers** and **project managers** in understanding the code structure and facilitating team coordination.

6.3.1 Component diagram

Searcher, importer and inferer's Component Diagram

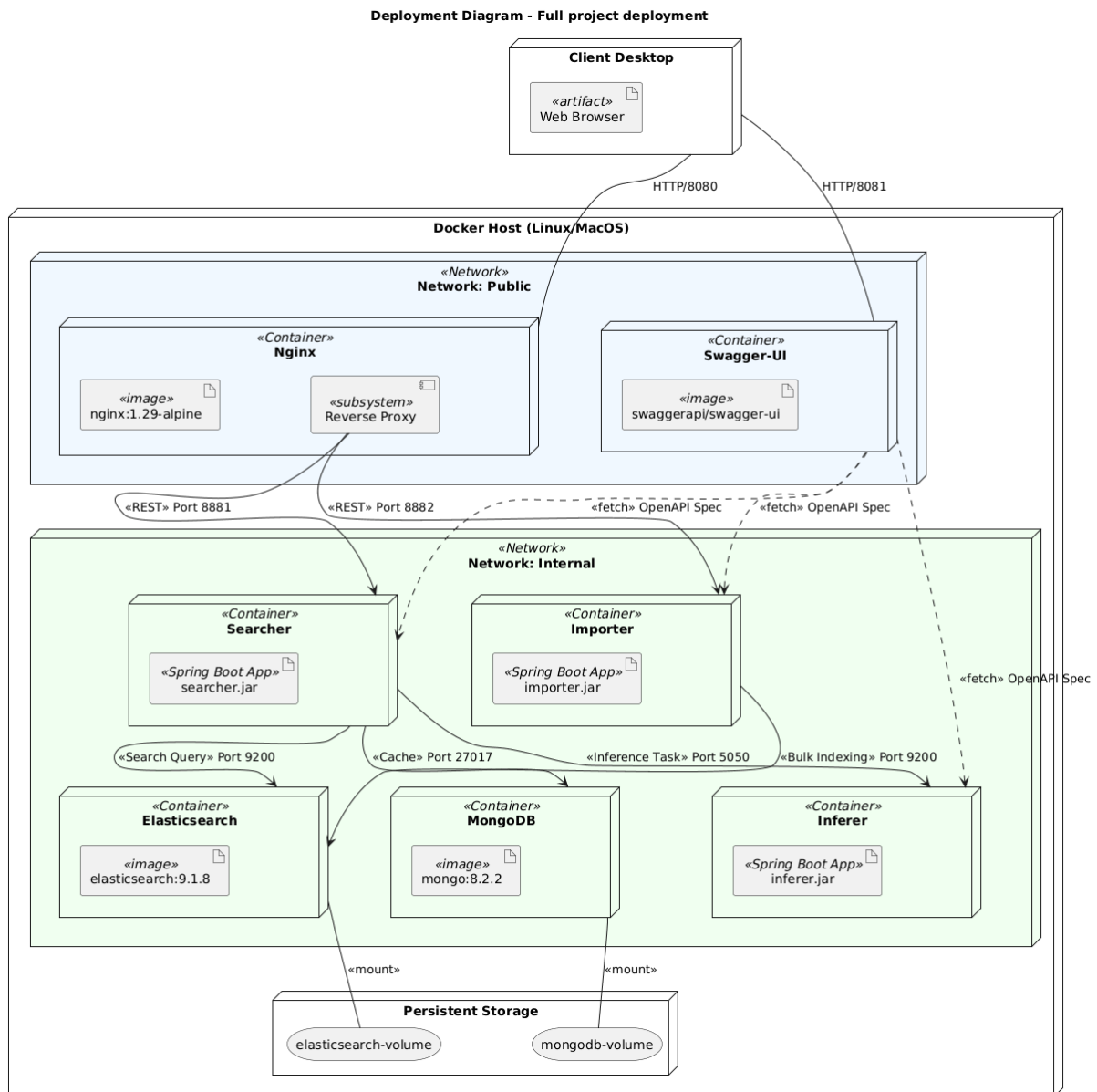


6.3.2 Package diagram



6.4 Deployment View

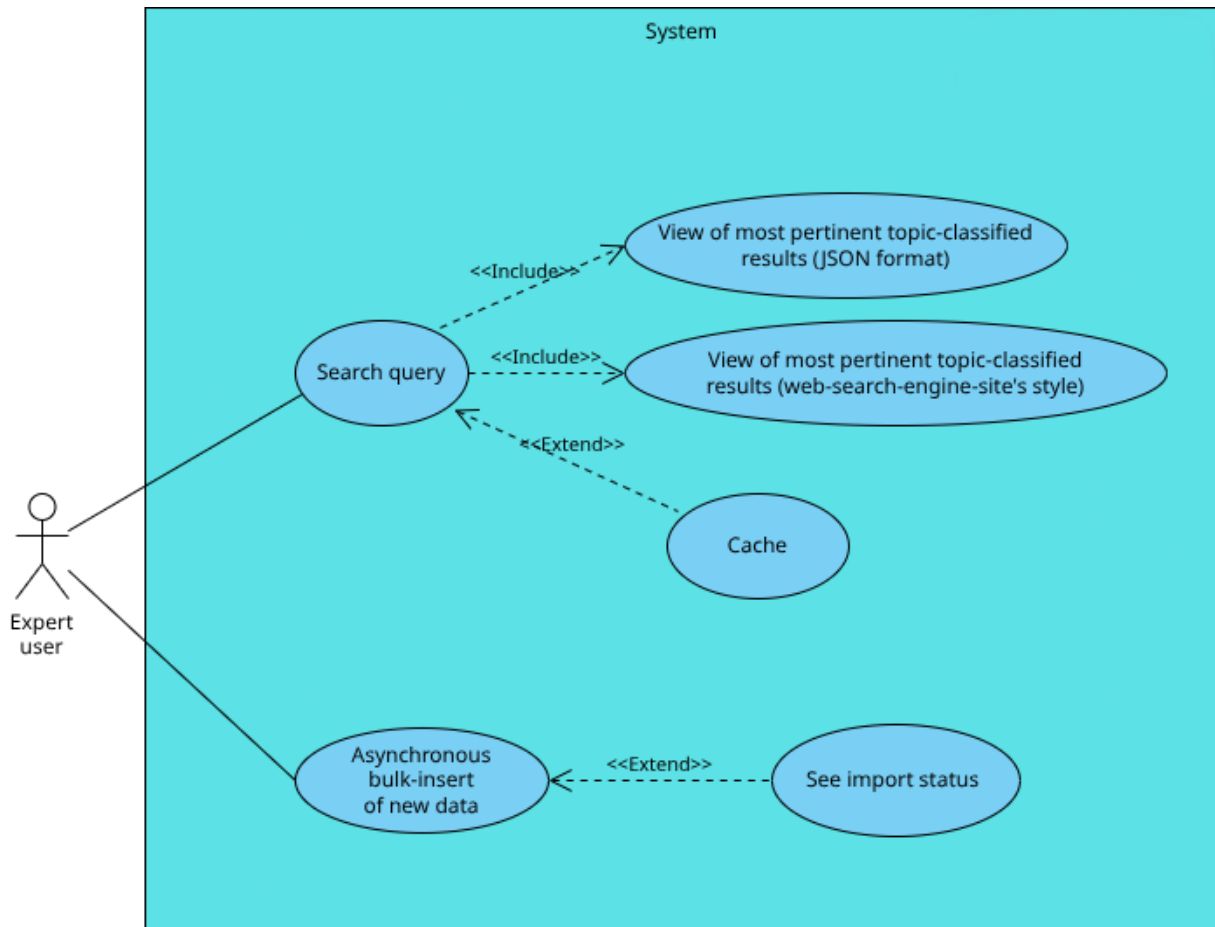
It shows the **physical view** of how software components are distributed across the hardware infrastructure.



6.4.1 Use Case View

It captures system functionality from the end-user perspective and serves as a unifying element that validates the other four views.

6.4.2 Use case diagram



6.4.3 User stories

The following user stories are written in a standard Agile format: **As a ... I want ... so that ...**. They align directly with the project's tasks and the professor's guidance.

- **User story 1 – Text document search**

As an expert user, I want to search documents using a free-text query so that I can quickly retrieve relevant documents from multiple online sources.

- **User story 2 – Searching multiple sources**

As an expert user, I want the system to search multiple document collections simultaneously so that I can obtain a broader set of results.

- **User story 3 – Ranking based on relevance**

As an expert user, I want documents to be ranked with more importance given to titles than to full text so that the most relevant results appear first.

- **User story 4 – Retrieving results from the cache**

As an expert user, I want previous searches to return instant results so that I can improve productivity.

- **User story 5 – Semantic enrichment of documents**

As an expert user, I want retrieved documents to be enriched with inferred topics to better understand their content.

- **User story 6 – Configurability of external services**

As a system administrator, I want to configure external service endpoints via application properties so that deployments can vary without code changes.

- **User story 7 – Exposure of a REST API**

As a developer, I want to access search functionality via a REST API for easy integration.

- **User story 8 – Error handling**

As an expert user, I want clear error responses when failures occur.

- **User story 9 – Components are up and running**

As a developer, I want to verify that the search service is operational.

- **User story 10 – Not an exact match**

As an expert user, I want approximate search results when no exact match exists.

- **User story 11 – Independent services**

As a DevOps engineer, I want to deploy services independently.

7 Architecture Justification

7.1 Purpose and Context

The system is designed to support large-scale analysis of textual data (millions of documents) originating from heterogeneous sources. Its primary users are expert analysts who require expressive full-text search, reproducible analytical workflows, and topic-based exploration with response times on the order of seconds. The system is deployed in a controlled, on-premises environment and prioritizes modularity and experimentation over consumer-grade latency.

7.2 Architectural Overview

The architecture follows a modular microservices pipeline:

- Custom Importer ingests and normalizes source data.
- Elasticsearch serves as the primary full-text search backend.
- NGINX acts as a reverse proxy in front of the UI and APIs.
- Query Handling Service retrieves cached results from MongoDB or executes new searches against Elasticsearch.
- Topic Modeling Service (MALLET) performs analytical enrichment.
- Query results and analytical outputs are cached for reuse.
- Swagger generates documentation for each microservice’s API.

This separation of concerns ensures scalability, maintainability, and analytical flexibility.

7.3 Roles and Justifications of Core Design Choices

7.3.1 Elasticsearch as Search Backend

Elasticsearch was chosen due to its ability to efficiently index and query millions of unstructured documents. Its inverted index, ranking capabilities, and aggregation support make it well-suited for exploratory full-text search. Given that query latency of a few seconds is acceptable, Elasticsearch provides an optimal balance between performance and expressiveness for expert users.

7.3.2 MongoDB for Query Caching and Analytical Results

MongoDB is used to store cached queries and derived analytical outputs. As a document-oriented database, it allows flexible schemas that accommodate evolving query definitions, topic modeling parameters, and metadata. This choice supports rapid experimentation and avoids rigid coupling between cached results and the underlying search index structure.

7.3.3 MALLET for Topic Modeling

MALLET was selected as the topic modeling engine due to its status as an academic standard. It provides well-understood, reproducible topic models suitable for research and expert analysis. Decoupling topic modeling from search allows analytical methods to evolve independently of retrieval infrastructure.

7.3.4 Custom Importer

A custom importer enables controlled preprocessing, normalization, and enrichment of heterogeneous data sources. In a multi-node cluster isolating write-heavy ingestion from read-heavy querying, the system maintains search performance while allowing re-ingestion or re-indexing without impacting downstream services.

7.3.5 Reverse Proxy and Controlled Exposure

NGINX provides a single, secure entry point to the system. Backend services are not directly exposed, and API ports remain closed within the internal network. This design reduces the attack surface in an on-premises deployment.

7.3.6 Swagger API generated docs

Swagger is used to generate the API documentation for each of the microservices that implement a controller. The Swagger microservice retrieves the information from each microservice and generates a webpage containing all the API documentation. This provides a clear separation of concerns, as well as a unique place to consult the API documentation.

The API endpoints described in the documentation refer to the internal private network used for inter-service communication. While explicitly exposing internal network addresses in the documentation may appear to conflict with security best practices, this choice is intentional and limited to documentation purposes only. It is adopted to clearly describe and document the system architecture and service interactions and does not reflect the actual exposure of these endpoints in a production environment. In fact, once the user understands how the application works, removing Swagger from the running environment is very easy: the entire service can be commented out in Docker Compose, and the call within the Spring project is explicit and compartmentalized in a specific class in each module.

7.3.7 Applying the Inference Service After the Search Phase

In the presented architecture, the inference service is deliberately applied after the execution of the search query, rather than during or before the document ingestion phase. This design choice is motivated by both architectural and computational considerations.

- From a computational efficiency perspective, inference operations are typically expensive in terms of time and resources. Applying inference to all documents at import time would require processing the entire corpus, regardless of whether the documents are ever queried. By contrast, performing inference only on the subset of documents returned by a search query ensures that computational resources are used on demand and only for documents that are actually relevant to user requests.

- From an architectural standpoint, decoupling search indexing (Elasticsearch) from semantic enrichment (MALLET) improves modularity and system flexibility. The search engine is responsible solely for fast, scalable retrieval based on textual relevance, while the inference service is treated as an independent step. This separation allows the inference model to be replaced, retrained, or reconfigured without requiring a full re-indexing of the document corpus.

Additionally, applying inference after the search phase enables query-dependent enrichment. The inference service can focus on extracting topics that are meaningful in the context of the specific query, rather than producing static ones at ingestion time. This approach is particularly advantageous in dynamic scenarios.

- From a performance and scalability perspective, this design reduces indexing latency and simplifies the ingestion pipeline. The system remains responsive even as the corpus grows since the cost of inference

scales with the number of retrieved results (fixed by us at 10 documents, but it can be modified) rather than with the total number of indexed documents.

This approach also aligns with the principles of lazy evaluation, in which expensive computations are performed only if strictly required. Combined with the caching mechanism implemented in the system, inferred results are persisted after the first computation, ensuring idempotency and low latency.

7.3.8 Custom trainer

The project includes the custom trainer used to train the MALLET topic modeling model. This choice was made to ensure transparency and reproducibility of the analytical workflow. By providing the trainer as part of the source code, expert users are able to retrain the model using their own datasets and adjust preprocessing steps or modeling parameters according to their specific domain needs. This flexibility is particularly important in research-oriented contexts, where topic quality and interpretability depend strongly on the underlying corpus.

Including the trainer also guarantees consistency between training and inference, as the same data processing logic is reused. Training is treated as an offline, resource-intensive operation and is therefore decoupled from the runtime query pipeline. New models can be generated and deployed without affecting system availability or query performance, supporting experimentation while preserving system stability.

7.3.9 Separated Owilix Dockerfile

The Dockerfile used to build the Owilix image is separated from the main project Dockerfile to preserve modularity and optionality in the data ingestion process. The use of OpenWebIndex data is not a mandatory requirement for the system, as alternative datasets, such as the Guardian corpus, are already available and sufficient to populate the pipeline. Additionally, Owilix is not the only possible mechanism for acquiring OWI data; users may choose to directly download datasets from the Open Web Index website, filter, and ingest them using the custom scripts and importer.

By isolating the Owilix Dockerfile, the architecture avoids introducing unnecessary dependencies for users who do not require OWI data or who prefer alternative data acquisition workflows. This separation simplifies deployment, reduces build complexity, and aligns with the system's experimentation-oriented design philosophy. Detailed instructions for using Owilix, when chosen, are provided in the project README. This option ensures modularity and prevents a fixed data ingestion strategy.

7.3.10 Data duplication

In our use case, we decided to use MongoDB only as a caching system to return inferred documents to the user and not as the main database system. This choice stems from the nature of the problem we needed to address. When studying the nature of Elasticsearch, at first glance, one might think that using Elasticsearch as a database is a misuse. Let's clarify the issue. It is true that Elasticsearch is not a relational database, a problem amplified by the indexing and search mechanisms that enable Elasticsearch to function. The operations that cause the most problems are updates. In Elastic, they are not a single operation:

1. Read the current document `_source`;
2. Merge the current `_source` with the new document;
3. Index the result of step 2;
4. Mark the original document as deleted. The deletion may not be immediate: it is marked as such, but until Lucene merges the indexes, the document remains. Even refreshing to ensure that only the latest documents are displayed is a very costly operation in Elasticsearch, slowing down ongoing update operations.

It is precisely this brief but very important concept that led us to our architecture. Our system is designed around an almost append-only data model since documents are filtered by URL; therefore, there can never be two or more identical ones. Since it is a web page search application, it is unlikely that we will need to make changes to the web pages. This does not mean that updates cannot be made in Elasticsearch; only that they should be done sparingly and, in the event of major changes,

rely on services (e.g., mongoDB, which is active and ready to use in its container) designed and intended to handle large amounts of data efficiently. This careful study has enabled us to avoid maintaining a full duplicate copy of indexed documents in MongoDB solely for the purpose of later ingestion into Elasticsearch, ensuring that we have more resources available for other services and do not have to manage updates to maintain data consistency between services, service failures, and all the other issues that can arise with duplicate and redundant data.

7.4 Architectural Benefits

- **Scalability:** search and analysis workloads are isolated and can be scaled independently.
- **Modularity:** each component can be replaced or extended with minimal impact.
- **Experimentation:** flexible schemas and cached analytical results support iterative research workflows. Additionally, alternatives are provided to customize the usage.
- **Fault Isolation:** failures in non-essential services do not compromise ingestion or search.
- **Reproducibility:** cached queries and topic models enable consistent analytical results.

7.5 Deployment Considerations

All services are containerized using Docker to ensure reproducibility and ease of deployment. The system is deployed on-premises to maintain control over data and infrastructure. Resource-intensive components such as Elasticsearch and topic modeling services are isolated to prevent contention. Security is enforced through network segmentation, with a reverse proxy as the only access point to the internal network and minimal service exposure.

7.6 Conclusion

This architecture intentionally prioritizes analytical rigor, modularity, and controlled scalability. By combining proven search technology, flexible data storage, and academically established modeling tools, the system provides a robust foundation for expert-driven exploration of large textual corpora while remaining adaptable to future research and technical evolution.

8 Our Project vs. the Competition

By comparing ourselves with our colleagues during the course of the project, we were able to analyze and understand their *modus operandi*. Our discussions revealed several points where our project differs from the competition. Here, we will list the main ones.

- As far as we know, we are the only group that seriously considered and managed to integrate OWI into the project. The other groups' reasoning was that "it took too much time to get it working and the documentation was out of date". As good students who believe in the ideals of open source and research, we contacted a professor involved in the project (we will not name him for privacy reasons) and, by providing our logs and our collaboration, we have arrived at a service that, at the time of writing this document (06/01/2026), is fully functional. Specifically, the dataset we provided is a subset of a 35GB dataset filtered and exported to JSON. We decided to share this information with our colleagues in the hope that they too will choose to use this service, given the motivation and effort that researchers are putting into developing it.
- We know that code is only part of a good project. While many of our colleagues have limited themselves to javadoc documentation, we provide users with not only all the javadoc documentation for the entire project, but also graphs and diagrams to help them understand our project intuitively and as fully as possible! We have based our approach on the "4+1 view model", providing graphs for expert users and programmers as well as consumer users.

- The project brief explicitly stated that the application should be used by an expert user. Although an expert user is not defined (is it a programmer who is familiar with the terminal or someone who prefers to use graphical tools?), we tried to accommodate both. As stated in the documentation, searches on imported datasets can be performed either from a browser via a page very similar to Google’s search page (used as inspiration to improve the usability and familiarity of the product) or from a terminal using, for example, the curl command. Even users who are not familiar with the terminal can use our application. The README is copy-and-paste ready, and after installation and import, the application is ready to be used via the GUI.
- The result is also accommodating for all types of expert users! From the terminal, you will get the results in JSON, while from the GUI you can get both pages filtered with Google-style topics and the JSON response that can be copied and pasted using the browser’s ‘Inspect’ tool (tested on Chromium-based browsers; on Firefox-based browsers, the procedure is a little longer because you have to analyse the response packet, but it is fully functional, covering almost all types of browser technology).
- We’ve thought of everything! In our project, you’ll find all the scripts you need in the scripts folder, all documented and ready to use, allowing you to install the application and get to work in less than 20 minutes¹, whether you’re a programmer or a regular user!
- As computer engineers, we know that we often find ourselves performing the same searches multiple times. To offer our users a better experience, we have introduced a caching system that allows for almost instantaneous response times after performing a search for the first time.
- Some of our colleagues mocked us for our simple, clean interface, showing us their polished and aesthetically appealing front ends. When we asked them how they had created those beautiful interfaces, they replied, ‘We used Gemini and React.js.’ After some discussion, we exposed the critical security flaws discovered this week in the library (more here [React.dev](#) and suggested caution until more was known, as the research team does not rule out other similar vulnerabilities. We decided to keep things simple and functional, avoiding unnecessary frills and remaining faithful to well-established and stable patterns (reverse proxy with static pages). We hope that our colleagues have taken the necessary countermeasures to manage the security of users who will use their application. We really hope so.
- Regarding the previous point, we use Nginx, the most popular reverse proxy. All incoming and outgoing traffic passes through this service, and there is no way to connect to internal services without going through it. Our colleagues, on the other hand, decided to cut corners by leaving this aspect out, leaving doors open (e.g. binding in docker compose files) and thus (probably) allowing users to access every service, bypassing any encapsulation and compartmentalisation constraints.

Popularity of Nginx in April 2025. As of April 2025, W3Tech’s web server count of all websites ranked Nginx first with 33.8%. Apache was second at 26.4% and Cloudflare Server third at 23.4%. See more on [Wikipedia](#).

¹Time calculated using an 80Mb/s network and an average computer; times may vary depending on many factors.